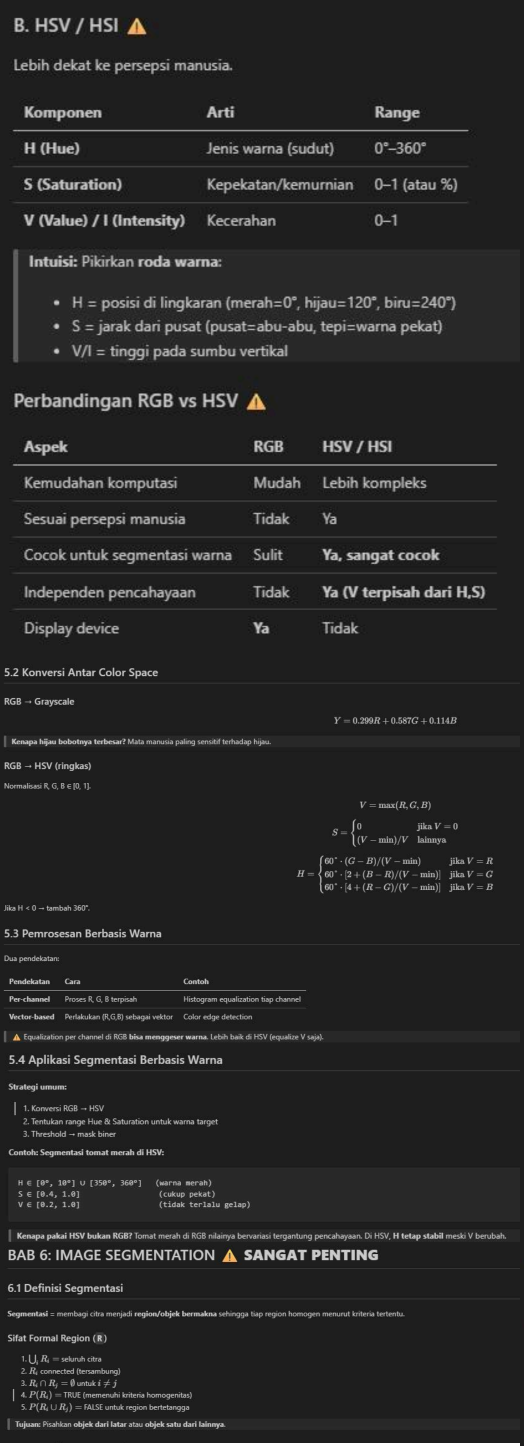


B. HSV / HSI ⚠

Lebih dekat ke persepsi manusia.

Komponen	Arti	Range
H (Hue)	Jenis warna (sudut)	0°–360°
S (Saturation)	Kepekatan/kemurnian	0–1 (atau %)
V (Value) / I (Intensity)	Kecerahan	0–1

> **Intuisi:** Pikirkan **roda warna:**
>
> - H = posisi di lingkaran (merah=0°, hijau=120°, biru=240°)
> - S = jarak dari pusat (pusat=abu-abu, tepi=warna pekat)
> - V/I = tinggi pada sumbu vertikal

Perbandingan RGB vs HSV ⚠

Aspek	RGB	HSV / HSI
Kemudahan komputasi	Mudah	Lebih kompleks
Sesuai persepsi manusia	Tidak	Ya
Cocok untuk segmentasi warna	Sulit	**Ya, sangat cocok**
Independen pencahayaan	Tidak	**Ya (V terpisah dari H,S)**
Display device	**Ya**	Tidak

5.2 Konversi Antar Color Space

RGB → Grayscale

$$Y = 0.299R + 0.587G + 0.114B$$

> **Kenapa hijau bobotnya terbesar?** Mata manusia paling sensitif terhadap hijau.

RGB → HSV (ringkas)

Normalisasi R, G, B ∈ [0, 1].

$$V = \max(R, G, B)$$

$$S = \begin{cases} 0 & \text{jika } V = 0 \\ (V - \min)/V & \text{lainnya} \end{cases}$$

$$H = \begin{cases} 60^\circ \cdot (G - B)/(V - \min) & \text{jika } V = R \\ 60^\circ \cdot [2 + (B - R)/(V - \min)] & \text{jika } V = G \\ 60^\circ \cdot [4 + (R - G)/(V - \min)] & \text{jika } V = B \end{cases}$$

Jika H < 0 → tambah 360°.

5.3 Pemrosesan Berbasis Warna

Dua pendekatan:

Pendekatan	Cara	Contoh
Per-channel	Proses R, G, B terpisah	Histogram equalization tiap channel
Vector-based	Perlakukan (R,G,B) sebagai vektor	Color edge detection

> ⚠ Equalization per channel di RGB **bisa menggeser warna**. Lebih baik di HSV (equalize V saja).

5.4 Aplikasi Segmentasi Berbasis Warna

Strategi umum:

1. Konversi RGB → HSV
2. Tentukan range Hue & Saturation untuk warna target
3. Threshold → mask biner

Contoh: Segmentasi tomat merah di HSV:

```
H ∈ [0°, 10°] ∪ [350°, 360°]   (warna merah)
S ∈ [0.4, 1.0]                  (cukup pekat)
V ∈ [0.2, 1.0]                  (tidak terlalu gelap)
```

> **Kenapa pakai HSV bukan RGB?** Tomat merah di RGB nilainya bervariasi tergantung pencahayaan. Di HSV, **H tetap stabil** meski V berubah.

BAB 6: IMAGE SEGMENTATION ⚠ **SANGAT PENTING**

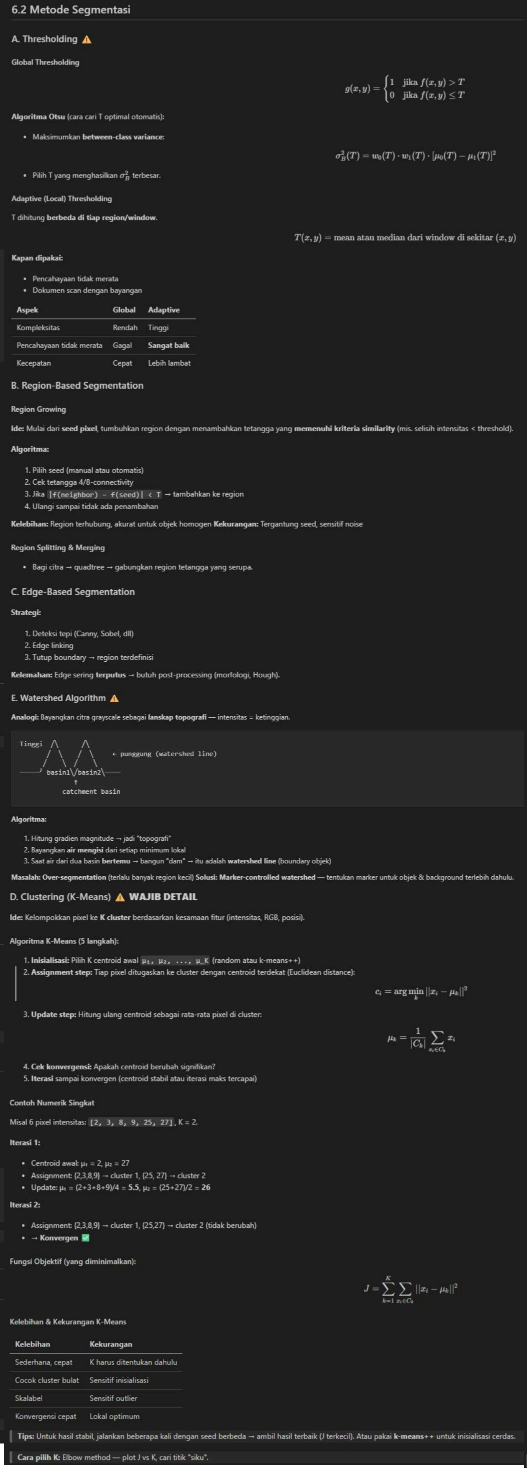
6.1 Definisi Segmentasi

Segmentasi = membagi citra menjadi **region/objek bermakna** sehingga tiap region homogen menurut kriteria tertentu.

Sifat Formal Region (`R`)

1. $\bigcup_i R_i$ = seluruh citra
2. R_i connected (tersambung)
3. $R_i \cap R_j = \emptyset$ untuk $i \neq j$
4. $P(R_i)$ = TRUE (memenuhi kriteria homogenitas)
5. $P(R_i \cup R_j)$ = FALSE untuk region bertetangga

> **Tujuan:** Pisahkan **objek dari latar** atau **objek satu dari lainnya**.



6.2 Metode Segmentasi

A. Thresholding ⚠

Global Thresholding

$$g(x, y) = \begin{cases} 1 & \text{jika } f(x, y) > T \\ 0 & \text{jika } f(x, y) \leq T \end{cases}$$

Algoritma Otsu (cara cari T optimal otomatis):

- Maksimumkan **between-class variance:**

$$\sigma_B^2(T) = w_0(T) \cdot w_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$$

- Pilih T yang menghasilkan σ_B^2 terbesar.

Adaptive (Local) Thresholding

T dihitung **berbeda di tiap region/window.**

$$T(x, y) = \text{mean atau median dari window di sekitar } (x, y)$$

Kapan dipakai:

- Pencahayaan tidak merata
- Dokumen scan dengan bayangan

Aspek	Global	Adaptive
Kompleksitas	Rendah	Tinggi
Pencahayaan tidak merata	Gagal	**Sangat baik**
Kecepatan	Cepat	Lebih lambat

B. Region-Based Segmentation

Region Growing

Ide: Mulai dari **seed pixel**, tumbuhkan region dengan menambahkan tetangga yang **memenuhi kriteria similarity** (mis. selisih intensitas < threshold).

Algoritma:

1. Pilih seed (manual atau otomatis)
2. Cek tetangga 4/8-connectivity
3. Jika `|f(neighbor) - f(seed)| < T` → tambahkan ke region
4. Ulangi sampai tidak ada penambahan

Kelebihan: Region terhubung, akurat untuk objek homogen **Kekurangan:** Tergantung seed, sensitif noise

Region Splitting & Merging

- Bagi citra → quadtree → gabungkan region tetangga yang serupa.

C. Edge-Based Segmentation

Strategi:

1. Deteksi tepi (Canny, Sobel, dll)
2. Edge linking
3. Tutup boundary → region terdefinisi

Kelemahan: Edge sering **terputus** → butuh post-processing (morfologi, Hough).

E. Watershed Algorithm ⚠

Analogi: Bayangkan citra grayscale sebagai **lanskap topografi** — intensitas = ketinggian.

```
Tinggi  /\     /\
       /  \   /  \      ← punggung (watershed line)
      /    \ /    \
─────┘ basin1\/basin2\────
              ↑
        catchment basin
```

Algoritma:

1. Hitung gradien magnitude → jadi "topografi"
2. Bayangkan **air mengisi** dari setiap minimum lokal
3. Saat air dari dua basin **bertemu** → bangun "dam" → itu adalah **watershed line** (boundary objek)

Masalah: Over-segmentation (terlalu banyak region kecil) **Solusi: Marker-controlled watershed** — tentukan marker untuk objek & background terlebih dahulu.

D. Clustering (K-Means) ⚠ **WAJIB DETAIL**

Ide: Kelompokkan pixel ke **K cluster** berdasarkan kesamaan fitur (intensitas, RGB, posisi).

Algoritma K-Means (5 langkah):

1. **Inisialisasi:** Pilih K centroid awal `μ₁, μ₂, ..., μ_K` (random atau k-means++)
2. **Assignment step:** Tiap pixel ditugaskan ke cluster dengan centroid terdekat (Euclidean distance):

$$c_i = \arg\min_k ||x_i - \mu_k||^2$$

3. **Update step:** Hitung ulang centroid sebagai rata-rata pixel di cluster:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

4. **Cek konvergensi:** Apakah centroid berubah signifikan?
5. **Iterasi** sampai konvergen (centroid stabil atau iterasi maks tercapai)

Contoh Numerik Singkat

Misal 6 pixel intensitas: `[2, 3, 8, 9, 25, 27]`, K = 2.

Iterasi 1:

- Centroid awal: $\mu_1 = 2$, $\mu_2 = 27$
- Assignment: {2,3,8,9} → cluster 1, {25, 27} → cluster 2
- Update: μ_1 = (2+3+8+9)/4 = **5.5**, μ_2 = (25+27)/2 = **26**

Iterasi 2:

- Assignment: {2,3,8,9} → cluster 1, {25,27} → cluster 2 (tidak berubah)
- → **Konvergen** ✅

Fungsi Objektif (yang diminimalkan):

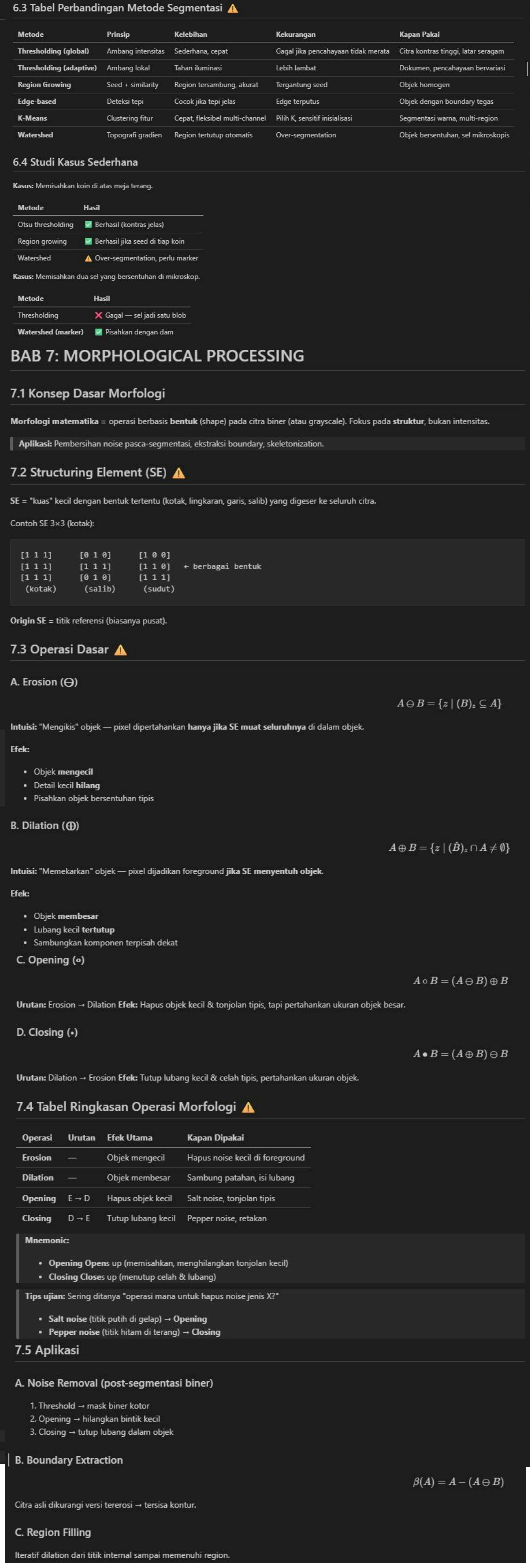
$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

Kelebihan & Kekurangan K-Means

Kelebihan	Kekurangan
Sederhana, cepat	K harus ditentukan dahulu
Cocok cluster bulat	Sensitif inisialisasi
Skalabel	Sensitif outlier
Konvergensi cepat	Lokal optimum

> **Tips:** Untuk hasil stabil, jalankan beberapa kali dengan seed berbeda → ambil hasil terbaik (J terkecil). Atau pakai **k-means++** untuk inisialisasi cerdas.

> **Cara pilih K:** Elbow method — plot J vs K, cari titik "siku".



6.3 Tabel Perbandingan Metode Segmentasi ⚠

Metode	Prinsip	Kelebihan	Kekurangan	Kapan Pakai
Thresholding (global)	Ambang intensitas	Sederhana, cepat	Gagal jika pencahayaan tidak merata	Citra kontras tinggi, latar seragam
Thresholding (adaptive)	Ambang lokal	Tahan iluminasi	Lebih lambat	Dokumen, pencahayaan bervariasi
Region Growing	Seed + similarity	Region tersambung, akurat	Tergantung seed	Objek homogen
Edge-based	Deteksi tepi	Cocok jika tepi jelas	Edge terputus	Objek dengan boundary tegas
K-Means	Clustering fitur	Cepat, fleksibel multi-channel	Pilih K, sensitif inisialisasi	Segmentasi warna, multi-region
Watershed	Topografi gradien	Region tertutup otomatis	Over-segmentation	Objek bersentuhan, sel mikroskopis

6.4 Studi Kasus Sederhana

Kasus: Memisahkan koin di atas meja terang.

Metode	Hasil
Otsu thresholding	✅ Berhasil (kontras jelas)
Region growing	✅ Berhasil jika seed di tiap koin
Watershed	⚠ Over-segmentation, perlu marker

Kasus: Memisahkan dua sel yang bersentuhan di mikroskop.

Metode	Hasil
Thresholding	❌ Gagal — sel jadi satu blob
Watershed (marker)	✅ Pisahkan dengan dam

BAB 7: MORPHOLOGICAL PROCESSING

7.1 Konsep Dasar Morfologi

Morfologi matematika = operasi berbasis **bentuk** (shape) pada citra biner (atau grayscale). Fokus pada **struktur**, bukan intensitas.

> **Aplikasi:** Pembersihan noise pasca-segmentasi, ekstraksi boundary, skeletonization.

7.2 Structuring Element (SE) ⚠

SE = "kuas" kecil dengan bentuk tertentu (kotak, lingkaran, garis, salib) yang digeser ke seluruh citra.

Contoh SE 3×3 (kotak):

```
[1 1 1]    [0 1 0]    [1 0 0]
[1 1 1]    [1 1 1]    [1 1 0]   ← berbagai bentuk
[1 1 1]    [0 1 0]    [1 1 1]
 (kotak)    (salib)    (sudut)
```

Origin SE = titik referensi (biasanya pusat).

7.3 Operasi Dasar ⚠

A. Erosion (⊖)

$$A \ominus B = \{z \mid (B)_z \subseteq A\}$$

Intuisi: "Mengikis" objek — pixel dipertahankan **hanya jika SE muat seluruhnya** di dalam objek.

Efek:

- Objek **mengecil**
- Detail kecil **hilang**
- Pisahkan objek bersentuhan tipis

B. Dilation (⊕)

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\}$$

Intuisi: "Memekarkan" objek — pixel dijadikan foreground **jika SE menyentuh objek.**

Efek:

- Objek **membesar**
- Lubang kecil **tertutup**
- Sambungkan komponen terpisah dekat

C. Opening (∘)

$$A \circ B = (A \ominus B) \oplus B$$

Urutan: Erosion → Dilation **Efek:** Hapus objek kecil & tonjolan tipis, tapi pertahankan ukuran objek besar.

D. Closing (•)

$$A \bullet B = (A \oplus B) \ominus B$$

Urutan: Dilation → Erosion **Efek:** Tutup lubang kecil & celah tipis, pertahankan ukuran objek.

7.4 Tabel Ringkasan Operasi Morfologi ⚠

Operasi	Urutan	Efek Utama	Kapan Dipakai
Erosion	—	Objek mengecil	Hapus noise kecil di foreground
Dilation	—	Objek membesar	Sambung patahan, isi lubang
Opening	E → D	Hapus objek kecil	Salt noise, tonjolan tipis
Closing	D → E	Tutup lubang kecil	Pepper noise, retakan

> **Mnemonic:**
>
> - **Opening Opens** up (memisahkan, menghilangkan tonjolan kecil)
> - **Closing Closes** up (menutup celah & lubang)

> **Tips ujian:** Sering ditanya "operasi mana untuk hapus noise jenis X?"
>
> - **Salt noise** (titik putih di gelap) → **Opening**
> - **Pepper noise** (titik hitam di terang) → **Closing**

7.5 Aplikasi

A. Noise Removal (post-segmentasi biner)

1. Threshold → mask biner kotor
2. Opening → hilangkan bintik kecil
3. Closing → tutup lubang dalam objek

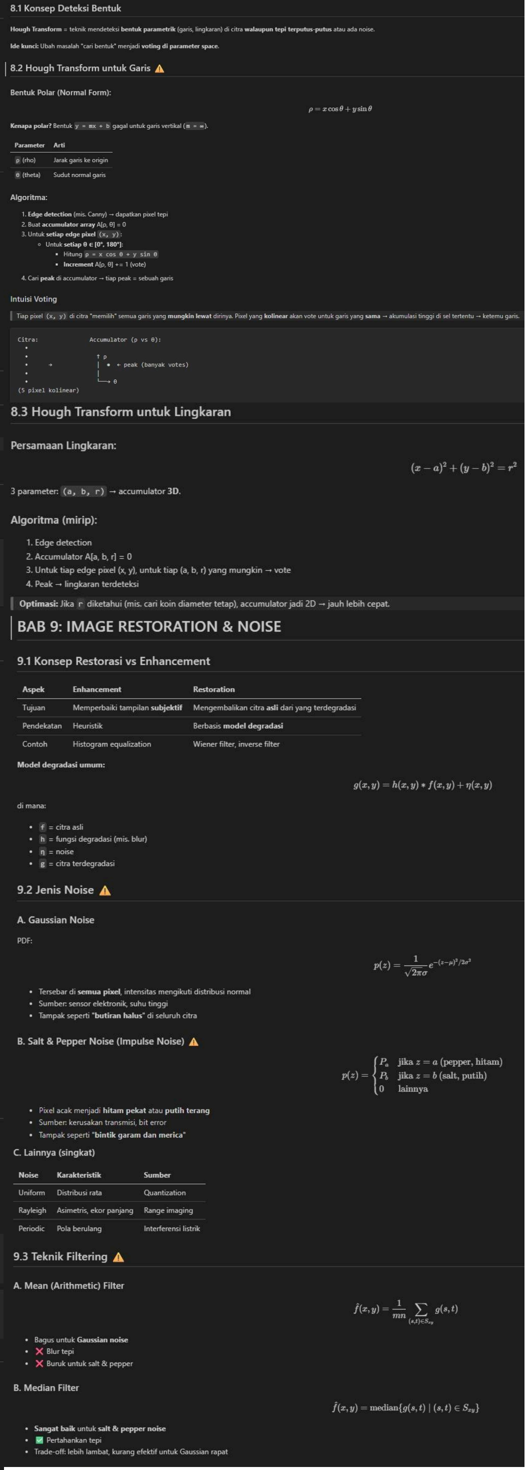
B. Boundary Extraction

$$\beta(A) = A - (A \ominus B)$$

Citra asli dikurangi versi tererosi → tersisa kontur.

C. Region Filling

Iteratif dilation dari titik internal sampai memenuhi region.



8.1 Konsep Deteksi Bentuk

Hough Transform = teknik mendeteksi **bentuk parametrik** (garis, lingkaran) di citra **walaupun tepi terputus-putus** atau ada noise.

Ide kunci: Ubah masalah "cari bentuk" menjadi **voting di parameter space.**

8.2 Hough Transform untuk Garis ⚠

Bentuk Polar (Normal Form):

$$\rho = x\cos\theta + y\sin\theta$$

Kenapa polar? Bentuk `y = mx + b` gagal untuk garis vertikal (`m = ∞`).

Parameter	Arti
`ρ` (rho)	Jarak garis ke origin
`θ` (theta)	Sudut normal garis

Algoritma:

1. **Edge detection** (mis. Canny) → dapatkan pixel tepi
2. Buat **accumulator array** A[ρ, θ] = 0
3. Untuk **setiap edge pixel** `(x, y)`:
    - Untuk **setiap θ ∈ [0°, 180°]:**
        - Hitung `ρ = x cos θ + y sin θ`
        - **Increment** A[ρ, θ] += 1 (vote)
4. Cari **peak** di accumulator → tiap peak = sebuah garis

Intuisi Voting

> Tiap pixel `(x, y)` di citra "memilih" semua garis yang **mungkin lewat** dirinya. Pixel yang **kolinear** akan vote untuk garis yang **sama** → akumulasi tinggi di sel tertentu → ketemu garis.

```
Citra:           Accumulator (ρ vs θ):
 •
 •                 ↑ ρ
 •    →            |  •  ← peak (banyak votes)
 •                 |
 •                 └──→ θ
(5 pixel kolinear)
```

8.3 Hough Transform untuk Lingkaran

Persamaan Lingkaran:

$$(x - a)^2 + (y - b)^2 = r^2$$

3 parameter: `(a, b, r)` → accumulator **3D.**

Algoritma (mirip):

1. Edge detection
2. Accumulator A[a, b, r] = 0
3. Untuk tiap edge pixel (x, y), untuk tiap (a, b, r) yang mungkin → vote
4. Peak → lingkaran terdeteksi

> **Optimasi:** Jika `r` diketahui (mis. cari koin diameter tetap), accumulator jadi 2D → jauh lebih cepat.

BAB 9: IMAGE RESTORATION & NOISE

9.1 Konsep Restorasi vs Enhancement

Aspek	Enhancement	Restoration
Tujuan	Memperbaiki tampilan **subjektif**	Mengembalikan citra **asli** dari yang terdegradasi
Pendekatan	Heuristik	Berbasis **model degradasi**
Contoh	Histogram equalization	Wiener filter, inverse filter

Model degradasi umum:

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$$

di mana:

- `f` = citra asli
- `h` = fungsi degradasi (mis. blur)
- `η` = noise
- `g` = citra terdegradasi

9.2 Jenis Noise ⚠

A. Gaussian Noise

PDF:

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\mu)^2/2\sigma^2}$$

- Tersebar di **semua pixel**, intensitas mengikuti distribusi normal
- Sumber: sensor elektronik, suhu tinggi
- Tampak seperti **"butiran halus"** di seluruh citra

B. Salt & Pepper Noise (Impulse Noise) ⚠

$$p(z) = \begin{cases} P_a & \text{jika } z = a \text{ (pepper, hitam)} \\ P_b & \text{jika } z = b \text{ (salt, putih)} \\ 0 & \text{lainnya} \end{cases}$$

- Pixel acak menjadi **hitam pekat** atau **putih terang**
- Sumber: kerusakan transmisi, bit error
- Tampak seperti **"bintik garam dan merica"**

C. Lainnya (singkat)

Noise	Karakteristik	Sumber
Uniform	Distribusi rata	Quantization
Rayleigh	Asimetris, ekor panjang	Range imaging
Periodic	Pola berulang	Interferensi listrik

9.3 Teknik Filtering ⚠

A. Mean (Arithmetic) Filter

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s, t)$$

- Bagus untuk **Gaussian noise**
- ❌ Blur tepi
- ❌ Buruk untuk salt & pepper

B. Median Filter

$$\hat{f}(x, y) = \text{median}\{g(s, t) \mid (s, t) \in S_{xy}\}$$

- **Sangat baik** untuk **salt & pepper noise**
- ✅ Pertahankan tepi
- Trade-off: lebih lambat, kurang efektif untuk Gaussian rapat